LAPORAN PRAKTIKUM PEKAN 3

STRUKTUR DATA

Stack

Disusun oleh:

Thaariq Salam

2511532022

Dosen Pengampu: Dr. Wahyudi S.T.M.T

Asisten Praktikum: Sherly Sukmadira

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

TAHUN 2026

KATA PENGANTAR

Puji syukur kehadirat Tuhan Yang Maha Esa atas rahmat-Nya sehingga penulis dapat menyelesaikan laporan praktikum Struktur Data mengenai materi "Stack". Laporan ini disusun untuk mendokumentasikan hasil pengerjaan serta pemahaman penulis terhadap mekanisme penyimpanan data LIFO (*Last In, First Out*) dalam pemrograman Java.

Melalui praktikum ini, penulis telah mempelajari berbagai cara mengimplementasikan *Stack*, mulai dari penggunaan array statis, pemanfaatan kelas ArrayList, hingga penggunaan kelas bawaan java.util.Stack.

Saya menyadari bahwa keberhasilan penyusunan laporan ini tidak lepas dari bimbingan dan dukungan berbagai pihak. Oleh karena itu, saya menyampaikan terima kasih yang sebesar-besarnya kepada:

1. Bapak Dr. Wahyudi S.T.M.T selaku Dosen Pengampu mata kuliah Struktur Data.
2. Kak Sherly Sukmadira selaku Asisten Praktikum yang telah memberikan arahan teknis selama pelaksanaan praktikum.

Saya menyadari sepenuhnya bahwa laporan praktikum ini masih jauh dari kata sempurna. Terdapat keterbatasan dalam analisis maupun teknis penulisan yang mungkin ditemukan. Oleh karena itu, saya sangat mengharapkan kritik yang membangun serta saran dari pembaca demi perbaikan di masa yang akan datang. Akhir kata, semoga laporan ini dapat memberikan manfaat, baik bagi saya pribadi maupun bagi pembaca yang ingin mendalami konsep pemrograman Java.

Padang, 19 April 2026

Thaariq Salam

DAFTAR ISI

KATA PENGANTAR.....2
DAFTAR ISI.....3
BAB I.....1
PENDAHULUAN.....1
1.1 Latar Belakang.....1
1.2 Tujuan.....1
1.3 Manfaat.....2
BAB II.....3
PEMBAHASAN.....3
2.1 Class latihanStack_2511532022.....3
2.1.1 Kode program.....3
2.1.2 Langkah Kerja.....3
2.1.3 Analisis hasil.....4
2.2 Class stackArray_2511532022 & Driver.....4
2.2.1 Kode program.....4
2.2.2 Langkah kerja.....6
2.2.3 Analisis hasil.....6
2.3 Class NilaiMaksimum_2511532022.....7
2.3.1 Kode program.....7
2.3.2 Langkah kerja.....7
2.3.3 Analisis hasil.....8
2.4 Class StackPostfix_2511532022.....8
2.4.1 Kode program.....8
2.4.2 Langkah kerja.....9
2.4.3 Analisis hasil.....10
2.5 Class SiswaStack_2511532022.....10
2.5.1 Kode program.....10
2.5.2 Langkah kerja.....12
2.5.3 Analisis hasil.....13
BAB III.....14
PENUTUP.....14
3.1 Kesimpulan.....14

TUGAS PEKAN 2.....15
1. Kode Program.....15
2. Penjelasan Kode: Website_2511532022.java.....17
3. Penjelasan Kode: Browser_2511532022.java.....18
3. Hasil Pengujian.....19

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam dunia informatika, pengelolaan data yang efisien merupakan kunci utama dalam pembangunan perangkat lunak yang optimal. Salah satu konsep dasar yang wajib dipahami adalah struktur data linier, di mana elemen-elemen data disusun secara berurutan. Di antara berbagai jenis struktur data, Stack (Tumpukan) menjadi sangat penting karena karakteristik uniknya yang menggunakan prinsip LIFO (Last In First Out), yakni data yang terakhir masuk akan menjadi yang pertama kali diproses atau dikeluarkan.

Penerapan Stack sangat luas dalam komputasi modern, mulai dari fitur *undo/redo* pada editor teks, pengelolaan alur eksekusi fungsi (call stack), hingga pemrosesan algoritma navigasi pada peramban web. Di lingkungan Java, Stack dapat diimplementasikan secara manual menggunakan array statis untuk efisiensi memori, maupun menggunakan ArrayList yang lebih dinamis dan fleksibel dalam menangani jumlah data yang berubah-ubah. Melalui praktikum ini, mahasiswa diharapkan tidak hanya memahami teori Stack secara abstrak, tetapi juga mampu mengimplementasikannya secara teknis dalam berbagai studi kasus, seperti evaluasi ekspresi postfix dan manajemen data objek terstruktur.

1.2 Tujuan

1. Memahami prinsip kerja struktur data Stack (LIFO).
2. Mampu mengimplementasikan Stack secara manual menggunakan array maupun secara dinamis menggunakan ArrayList.
3. Memahami cara penggunaan library java.util.Stack.
4. Mampu menerapkan Stack dalam pemecahan masalah (studi kasus postfix dan pencarian nilai maksimum).

1.3 Manfaat

1. Mahasiswa dapat memilih metode implementasi Stack yang paling efisien sesuai kebutuhan memori.
2. Meningkatkan kemampuan logika pemrograman dalam mengolah data linear.
3. Memberikan pemahaman mendalam tentang manipulasi data melalui metode push, pop, dan peek.

BAB II

PEMBAHASAN

2.1 Class latihanStack_2511532022

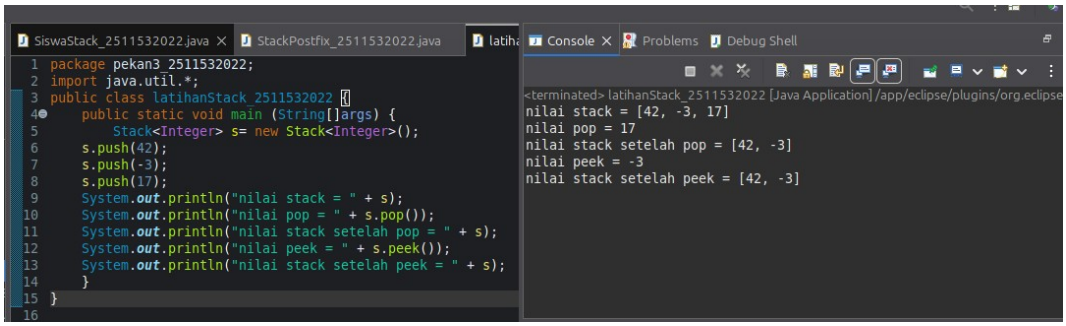
2.1.1 Kode program

```
package pekan3_2511532022;
import java.util.*;
public class latihanStack_2511532022 {
      public static void main (String[]args) {
      Stack<Integer> s= new Stack<Integer>();
      s.push(42);
      s.push(-3);
      s.push(17);
      System.out.println("nilai stack = " + s);
      System.out.println("nilai pop = " + s.pop());
      System.out.println("nilai stack setelah pop = " + s);
      System.out.println("nilai peek = " + s.peek());
      System.out.println("nilai stack setelah peek = " + s);
      }
}
```

2.1.2 Langkah Kerja

1) Instansiasi Stack<Integer> s = new Stack<Integer>().
2) Memasukkan angka 42, -3, dan 17 menggunakan s.push().
3) Mencetak seluruh isi stack, melakukan pop(), lalu mencetak kembali untuk melihat pengurangan elemen.
4) Menjalankan s.peek() dan mencetak stack kembali untuk membuktikan bahwa peek tidak mengurangi elemen.

2.1.3 Analisis hasil



Percobaan ini menunjukkan kemudahan penggunaan library java.util.Stack. Hasil di konsol memperlihatkan bahwa stack di Java secara otomatis mencetak seluruh elemen dalam format list [42, -3, 17], memudahkan programmer dalam proses debugging.

2.2 Class stackArray_2511532022 & Driver

2.2.1 Kode program

```
package pekan3_2511532022;

public class stackArray_2511532022{
    static final int MAX = 1000;
    int top;
    int a[] = new int[MAX];
    boolean isEmpty() {
        return (top < 0);
    }
    stackArray_2511532022() {
        top = -1;
    }
    boolean push(int x) {
        if (top >= (MAX - 1)) {
            System.out.println("Stack overflow");
            return false;
        } else {
            a[++top] = x;
            System.out.println(x + " masukkan ke dalam stack");
            return true;
```

```
        }
    }
    int pop() {
        if (top < 0) {
            System.out.println("Stack underflow");
            return 0;
        }
        else {
            int x = a[top--];
            return x;
        }
    }
    int peek() {
        if (top < 0) {
            System.out.println("Stack underflow");
            return 0;
        }
        else {
            int x = a[top];
            return x;
        }
    }
    void print(){
        for (int i = top; i >= 1; i--) {
            System.out.println(" " + a[i]);
        }
    }
}
```

Driver:

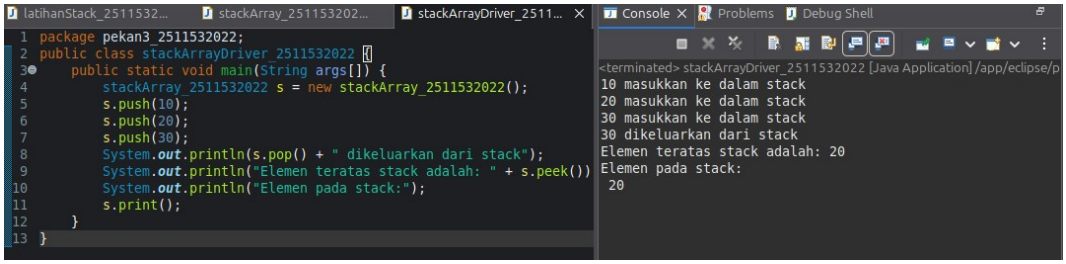
```
package pekan3_2511532022;
public class stackArrayDriver_2511532022 {
    public static void main(String args[]) {
        stackArray_2511532022 s = new stackArray_2511532022();
        s.push(10);
        s.push(20);
        s.push(30);
        System.out.println(s.pop() + " dikeluarkan dari stack");
```

```
        System.out.println("Elemen teratas stack adalah: " +
            s.peek());
        System.out.println("Elemen pada stack:");
        s.print();
    }
}
```

2.2.2 Langkah kerja

1) Menetapkan batas maksimal array (MAX = 1000) dan menginisialisasi variabel top dengan nilai -1 sebagai penanda stack kosong.
2) Method push(int x) terlebih dahulu mengecek apakah stack sudah penuh (top >= MAX - 1). Jika belum, nilai top dinaikkan dan array diisi.
3) Melakukan pengecekan underflow (jika top < 0). Method pop() mengambil nilai lalu menurunkan indeks top, sedangkan peek() hanya mengambil nilai tanpa mengubah indeks.
4) Method print() melakukan iterasi dari posisi top hingga indeks ke-1 untuk mencetak elemen ke layar.
5) Pada class stackArrayDriver, objek stack dibuat, diisi dengan angka 10, 20, 30, lalu diuji dengan operasi pop dan peek.

2.2.3 Analisis hasil



Implementasi ini menunjukkan manajemen stack pada level memori statis. Driver mengonfirmasi bahwa angka 30 berhasil dikeluarkan pertama kali. Namun, terdapat batasan pada method print() di mana data pada indeks 0 tidak tercetak karena perulangan berhenti di angka 1.

2.3 Class NilaiMaksimum_2511532022

2.3.1 Kode program

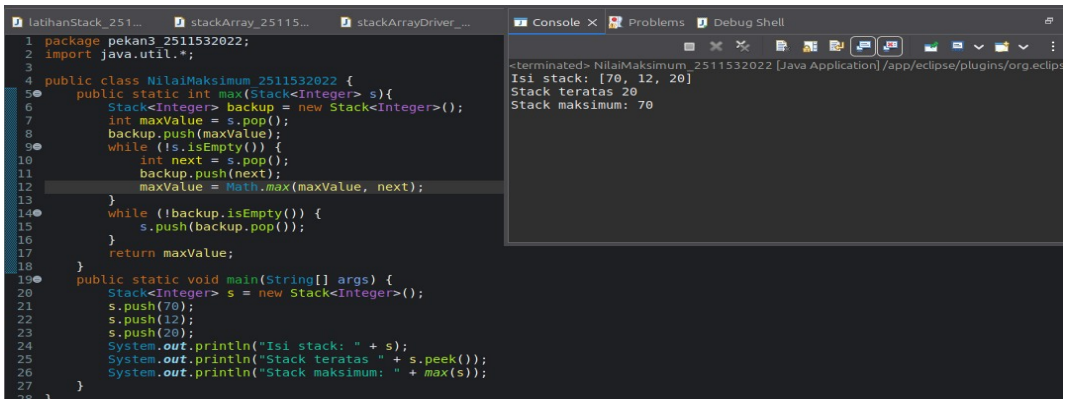
```
package pekan3_2511532022;
import java.util.*;

public class NilaiMaksimum_2511532022 {
    public static int max(Stack<Integer> s){
        Stack<Integer> backup = new Stack<Integer>();
        int maxValue = s.pop();
        backup.push(maxValue);
        while (!s.isEmpty()) {
            int next = s.pop();
            backup.push(next);
            maxValue = Math.max(maxValue, next);
        }
        while (!backup.isEmpty()) {
            s.push(backup.pop());
        }
        return maxValue;
    }
    public static void main(String[] args) {
        Stack<Integer> s = new Stack<Integer>();
        s.push(70);
        s.push(12);
        s.push(20);
        System.out.println("Isi stack: " + s);
        System.out.println("Stack teratas " + s.peek());
        System.out.println("Stack maksimum: " + max(s));
    }
}
```

2.3.2 Langkah kerja

1) Membuat stack backup dan menetapkan elemen pertama hasil pop() dari stack asli sebagai nilai maxValue awal.

2) Menjalankan loop while untuk mengeluarkan sisa elemen stack asli satu per satu.
3) Setiap elemen yang dikeluarkan dibandingkan dengan maxValue menggunakan Math.max(), lalu elemen tersebut disimpan ke dalam stack backup.
4) Memindahkan kembali seluruh isi dari stack backup ke stack asli menggunakan perulangan while (!backup.isEmpty()).

2.3.3 Analisis hasil



Program secara tepat mengidentifikasi nilai 70 sebagai angka terbesar dalam tumpukan. Penggunaan stack tambahan (backup) sangat penting secara prosedural agar data pada stack utama tidak hilang setelah proses pencarian selesai, menjaga integritas struktur data tersebut.

2.4 Class StackPostfix_2511532022

2.4.1 Kode program

```
package pekan3_2511532022;
import java.util.*; // Penting: Supaya Stack dan Scanner dikenali

public class StackPostfix_2511532022 {
   public static int postfixEvaluate_2511532022(String expression{
      Stack<Integer> s = new Stack<Integer>();
```

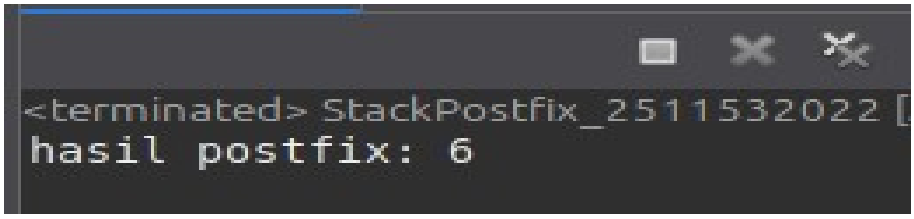
```
    Scanner input_2511532022 = new Scanner(expression);
    while (input_2511532022.hasNext()) {
          if (input_2511532022.hasNextInt()) {
              s.push(input_2511532022.nextInt());
          } else {
              String operator = input_2511532022.next();
              int operand2 = s.pop();
              int operand1_2511532022 = s.pop();

              if (operator.equals("+")) {
                  s.push(operand1_2511532022 + operand2);
              } else if (operator.equals("-")) {
                  s.push(operand1_2511532022 - operand2);
              } else if (operator.equals("*")) {
                  s.push(operand1_2511532022 * operand2);
              } else if (operator.equals("/")) {
                  s.push(operand1_2511532022 / operand2);
              }
          } // Penutup blok 'else' dipindah ke sin
    }
    input_2511532022.close();  //  Tutup  scanner  setelah  loop
    selesai
    return s.pop();
}

public static void main(String[] args) {
     // Contoh: "5 2 4 * + 7 -"
              //  Cara  bacanya:  5  +  (2  *  4)  -  7  =  6
    System.out.println("hasil        postfix:"        +
    postfixEvaluate_2511532022 ("5 2 4 * + 7 -"));
 }
```

2.4.2 Langkah kerja

1) Instansiasi Stack<Integer> untuk menyimpan nilai angka dan Scanner untuk memproses string ekspresi expression.
2) Menjalankan loop while (input.hasNext()) untuk memeriksa setiap komponen string.

3) Jika token adalah angka (hasNextInt()), maka nilai tersebut di-push ke dalam stack.
4) Jika ditemukan operator, program mengambil dua elemen teratas menggunakan pop() (sebagai operand2 dan operand1).
5) Menggunakan struktur if-else untuk mengeksekusi operasi (+, -, *, /) berdasarkan operator tersebut, lalu hasilnya di-push kembali ke stack.
6) Hasil akhir mengambil elemen terakhir yang tersisa di dalam stack sebagai hasil evaluasi.

2.4.3 Analisis hasil



Program berhasil memproses input "5 2 4 * + 7 -" dan menghasilkan output **6**. Secara teknis, stack digunakan sebagai memori sementara untuk menahan operand hingga bertemu operator yang sesuai, sehingga urutan pengerjaan matematika tetap akurat tanpa memerlukan tanda kurung.

2.5 Class SiswaStack_2511532022

2.5.1 Kode program

```
package pekan3_2511532022;
import java.util.ArrayList;

class Siswa {
	String nama;
	int nim;

	public Siswa(String nama, int nim) {
		this.nama = nama;
```

```
            this.nim = nim;
        }

        @Override
        public String toString() {
            return "NIM: " + nim + ", Nama: " + nama;
        }
    }
    public class SiswaStack_2511532022 {
        private ArrayList<Siswa> stack;

        public SiswaStack_2511532022 (){
            stack = new ArrayList<>();
        }

        public void push(Siswa mhs) {
            stack.add(mhs);
        }

        public Siswa pop() {
            if (!isEmpty()) {
                return stack.remove(stack.size() - 1);
            }
            return null;
        }

        public Siswa peek() {
            if (!isEmpty()) {
                return stack.get(stack.size() -1);
            }
            return null;
        }

        public boolean isEmpty() {
            return stack.isEmpty();
        }

        public void tampilkanSiswa() {
            for (int i= stack.size() - 1; i >= 0; i--) {
```

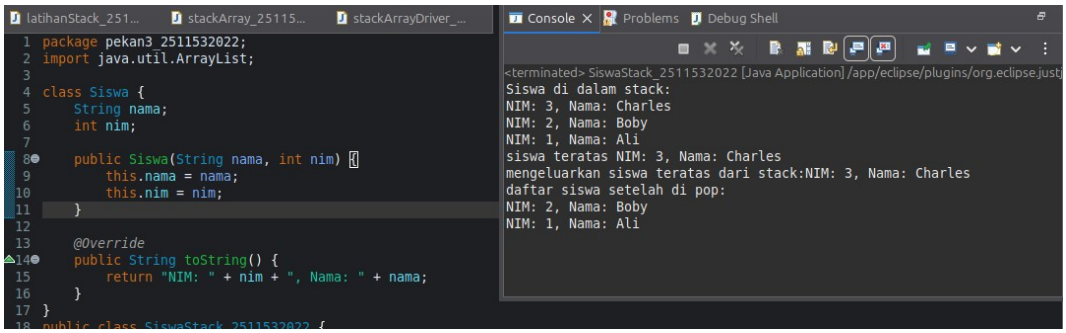
```
                System.out.println(stack.get(i));
            }
        }
        public static void main(String[] args) {
            SiswaStack_2511532022 studentStack = new
              SiswaStack_2511532022();

            Siswa mhs1 = new Siswa("Ali", 1);
            Siswa mhs2 = new Siswa("Boby", 2);
            Siswa mhs3 = new Siswa("Charles", 3);

            studentStack.push(mhs1);
            studentStack.push(mhs2);
            studentStack.push(mhs3);

            System.out.println("Siswa di dalam stack:");
            studentStack.tampilkanSiswa();

            System.out.println("siswa teratas " +
              studentStack.peek());
            System.out.println("mengeluarkan siswa teratas dari
              stack:" + studentStack.pop());
            System.out.println("daftar siswa setelah di pop: ");
            studentStack.tampilkanSiswa();
        }
}
```

2.5.2 Langkah kerja

1) Membuat class Siswa dengan atribut nama dan nim, dilengkapi konstruktor untuk inisialisasi data serta *method* toString() untuk memformat tampilan objek saat dicetak.
2) Di dalam class utama, mendeklarasikan atribut ArrayList<Siswa> stack sebagai wadah penyimpanan yang bersifat dinamis.
3) Membuat method push(Siswa mhs) yang memanggil fungsi stack.add(mhs) untuk menambahkan objek ke urutan terakhir list.

4) Membuat method pop() yang memeriksa kondisi stack; jika tidak kosong, elemen pada indeks terakhir (stack.size() - 1) akan dihapus dan dikembalikan.
5) Membuat method peek() untuk mengintip data terakhir dan isEmpty() untuk mengecek ketersediaan elemen dalam list.
6) Membuat method tampilkanSiswa() yang melakukan perulangan for secara terbalik (dari indeks terbesar ke 0) untuk menampilkan isi stack.
7) Uji Coba (Main) dengan melakukan instansiasi objek siswa, memasukkannya ke stack, lalu menjalankan operasi peek dan pop untuk memverifikasi alur kerja.

2.5.3 Analisis hasil



Implementasi ini menunjukkan fleksibilitas dalam menyimpan tipe data bentukan (objek). Berdasarkan hasil eksekusi, data "Charles" yang terakhir dimasukkan menjadi data pertama yang keluar saat operasi pop dijalankan. Hal ini membuktikan bahwa prinsip LIFO dapat dikelola dengan baik menggunakan indeks dinamis pada ArrayList.

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan serangkaian praktikum yang telah dilakukan, dapat disimpulkan bahwa struktur data Stack merupakan komponen fundamental dalam pemrograman yang secara konsisten menerapkan prinsip LIFO (Last In First Out). Melalui berbagai implementasi, terlihat bahwa Stack sangat fleksibel karena dapat dibangun menggunakan ArrayList untuk menangani data dinamis seperti objek siswa, maupun menggunakan array statis untuk kontrol memori yang lebih ketat namun terbatas pada kapasitas tertentu. Selain itu, praktikum ini membuktikan kegunaan riil Stack dalam logika komputasi, seperti kemampuannya mengevaluasi ekspresi matematika postfix tanpa bantuan tanda kurung serta kemampuannya mengolah data melalui mekanisme backup untuk menjaga integritas struktur asli. Pemahaman mendalam mengenai manipulasi indeks top dan penggunaan library standar Java memberikan wawasan bagi praktikan dalam memilih metode implementasi yang paling efisien sesuai dengan kompleksitas masalah yang dihadapi.

TUGAS PEKAN 2

Topik : Implementasi Stack (LIFO) menggunakan java.util.Stack

Tema : Simulasi Browser History

1. Kode Program

Kelas ADT : Website_2511532022.java

```
package pekan3_2511532022;

public class Website_2511532022 {
    // Variabel pake akhiran 4 digit NIM sesuai aturan
    private String judul_2022;
    private String url_2022;

    public Website_2511532022(String judul_2022, String url_2022{
        this.judul_2022 = judul_2022;
        this.url_2022 = url_2022;
    }

    // Method buat update data sekaligus
    public void setDataWeb_2022(String judul, String url) {
        this.judul_2022 = judul;
        this.url_2022 = url;
    }

    // Getter tetap ada satu-satu buat ambil data
    public String getJudul_2022() { return judul_2022; }
    public String getUrl_2022() { return url_2022; }

    // Biar gampang pas mau nampilin info web
    public void infoWeb() {
        System.out.println(judul_2022 + " (" + url_2022 + ")");
    }
}
```

Kelas Driver: Browser_2511532022.java

```
package pekan3_2511532022;

import java.util.Stack;
import java.util.Scanner;

public class Browser_2511532022 {
    public static void main(String[] args) {
```

```
// Inisialisasi stack buat simpan history
Stack<Website_2511532022> history = new Stack<>();
Scanner sc = new Scanner(System.in);
int menu;

do {
    System.out.println("\n=== Browser History NIM:
      2511532022 ===");
    System.out.println("1. Kunjungi Website (Push)");
    System.out.println("2. Tombol Back (Pop)");
    System.out.println("3. Lihat Halaman Aktif (Peek)");
    System.out.println("4. Cek Status History");
    System.out.println("5. Keluar");
    System.out.print("Pilih menu: ");
    menu = sc.nextInt();
    sc.nextLine(); // bersihin sisa enter biar ngga skip
      input string

    if (menu == 1) {
        System.out.print("Judul web: ");
        String jdl = sc.nextLine();
        System.out.print("URL: ");
        String link = sc.nextLine();

        // Masukin data baru ke tumpukan paling atas
        history.push(new Website_2511532022(jdl, link));
        System.out.println("-> Berhasil buka halaman
          baru.");

    } else if (menu == 2) {
        // Cek dulu jangan sampai pop pas stack kosong
            (biar ga error)
        if (history.isEmpty()) {
          System.out.println("-> History kosong, ga
            bisa mundur.");
        } else {
            Website_2511532022 hapus = history.pop();
            System.out.println("-> Back! Keluar dari: " +
              hapus.getJudul_2022());
        }

    } else if (menu == 3) {
        if (history.isEmpty()) {
            System.out.println("-> Ga ada halaman yang
              aktif.");
        } else {
            System.out.print("-> Kamu lagi buka: ");
            // Mengintip data paling atas tanpa hapus
            history.peek().infoWeb();
        }

    } else if (menu == 4) {
        // Cek kondisi history pake isEmpty() dan size()
        if (history.isEmpty()) {
            System.out.println("-> Riwayat masih kosong
              melompong.");
        } else {
```

```
                System.out.println("-> Ada " + history.size() + "
                  halaman di riwayat.");
                }

            } else if (menu == 5) {
                System.out.println("-> Bye! Browser ditutup.");
            } else {
                System.out.println("-> Menu itu ga ada, pilih
                  yang bener ya.");
            }

        } while (menu != 5);

        sc.close();
    }
}
```

2. Penjelasan Kode: Website_2511532022.java

Class ini bertindak sebagai ADT (Abstract Data Type) atau cetakan objek untuk setiap halaman web.

1. Atribut (judul_2022, url_2022)

   Variabel ini dideklarasikan sebagai private untuk menerapkan prinsip Enkapsulasi, artinya data tidak bisa diubah sembarangan dari luar class tanpa melalui *method* yang tersedia.

2. Constructor

   Baris ini otomatis berjalan saat kamu mengetik new Website_.... Fungsinya langsung memetakan nilai input ke dalam atribut class.

3. Method Setter (setDataWeb_2022)

   Berfungsi sebagai mutator. Jika di kemudian hari data website perlu diperbarui tanpa membuat objek baru, *method* ini yang akan dipanggil.

4. Method Getter (getJudul_2022, getUrl_2022)

   Karena atributnya private, *method* ini adalah satu-satunya cara bagi class Driver untuk mengambil teks judul atau URL untuk ditampilkan di layar.

5. Method infoWeb()

Sebuah fungsi pembantu (*helper*) untuk mencetak format judul dan URL dalam satu baris agar kode di class Driver lebih bersih.

3. Penjelasan Kode: Browser_2511532022.java

Class ini adalah Driver yang mengontrol logika jalannya program.

1. Stack<Website_2511532022> history = new Stack<>()

   Baris ini menciptakan struktur data tumpukan. Perhatikan bahwa tipe datanya bukan String atau Integer, melainkan objek Website_2511532022.

2. sc.nextLine() setelah nextInt()

   Ini baris krusial. nextInt() hanya mengambil angka, tapi menyisakan karakter "Enter" di memori. nextLine() tambahan ini berfungsi "menyapu" sisa enter tersebut agar input judul berikutnya tidak terlewat (*skipped*).

3. Logika Menu 1 (Push): Program meminta input teks dari pengguna.

   - history.push(new Website_...) menciptakan objek baru di memori dan langsung meletakkannya di posisi teratas tumpukan.

4. Logika Menu 2 (Pop):

   - Pengecekan isEmpty(): Baris terpenting untuk mencegah *error*. Jika tumpukan kosong dan kita paksa pop(), Java akan melempar EmptyStackException.
   - history.pop() mengambil objek paling atas dan menghapusnya dari riwayat.

5. Logika Menu 3 (Peek):

   history.peek() hanya mengambil referensi objek teratas tanpa menghapus. Ini merepresentasikan fitur "Halaman Aktif" pada browser.

3. Hasil Pengujian

```
<terminated> Browser_2511532022 [Java Application] /app/eclipse/plugins/org.eclipse.justj.openjdk.hotspot.jre.full.linux.x86

=== Browser History NIM: 2511532022 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Keluar
Pilih menu: 2
-> History kosong, ga bisa mundur.

=== Browser History NIM: 2511532022 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Keluar
Pilih menu: 1
Judul web: sso unand
URL: https://sso.unand.ac.id/login
-> Berhasil buka halaman baru.

=== Browser History NIM: 2511532022 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Keluar
Pilih menu: 1
Judul web: ilearn informatika
URL: https://it.ilearn.unand.ac.id/
-> Berhasil buka halaman baru.
```

```
<terminated> Browser_2511532022 [Java Application] /app/eclipse/plugins/org.eclipse.justj.openjdk.hotspot.jre.full.linu
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Keluar
Pilih menu: 3
-> Kamu lagi buka: ilearn informatika (https://it.ilearn.unand.ac.id/)

=== Browser History NIM: 2511532022 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Keluar
Pilih menu: 2
-> Back! Keluar dari: ilearn informatika

=== Browser History NIM: 2511532022 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Keluar
Pilih menu: 3
-> Kamu lagi buka: sso unand (https://sso.unand.ac.id/login)

=== Browser History NIM: 2511532022 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Keluar
Pilih menu: 4
-> Bye! Browser ditutup.
```